

Crypto challenge - Wiki Mersenne

deuterium — 2021-07-25

Canonical HTML: <https://deut-erium.github.io/2021/07/25/wiki-mersenne.html>

Abstract

Crypto challenge - Wiki Mersenne, published in deuterium's blog.

Numpy uses plain old implementation of Mersenne Twister as the default pseudo random number generation.

Now Mersenne Twister by itself is not so bad, what's bad is actually the seed initialization (i.e. deriving the initial state of the random number generator from the seed) same as explained in the [Wikipedia Implementation](#) which has been replaced in [Improved Initialization](#) with a more non-linear initialization with an array of seeds.

So, given a few (very few actually) outputs from the numpy random RNG, can you recover the flag?

Note that the seed is a 32-bit value and can be bruteforced easily, thats not the goal of this challenge, the goal of the challenge is to figure out a way which works equivalently well for MT-19937-64 bit as 64-bit is out of the bounds of bruteforce for a reasonably practical CTFer :P

```
import os
from numpy import random
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
from secret import flag

def rand_32():
    return int.from_bytes(os.urandom(4), 'big')

flag = pad(flag, 16)
random.seed(rand_32())
iv, key = random.bytes(16), random.bytes(16)
cipher = AES.new(key, iv=iv, mode=AES.MODE_CBC)
flag = iv + cipher.encrypt(flag)

print(flag.hex())

# output
# ba84b595a6c47ab8b5229df78b313fd983368f94c86e063ad9e60b53debf4cf062e0e7ee\
# 975a58ede95877add16603d089a7c01b5581278440b2fc8a25e698ae869a2c67de7b8a5e\
# bbe47fcb6cb210237d6cb60d06dabbf7756a2364ba2fbb5b0f0c04fb4383f66ff755c725\
# 2b699c33
```

Enter the flag

The local SHA-256 checker requires JavaScript. The challenge itself does not.

| Interactive controls are omitted from this PDF. See the canonical HTML version.